

# Proyecto de Inteligencia Artificial

## Jugador Autónomo para HEX

Josué Javier Senarega Claro

Grupo: C-311

March 2026

### Resumen

Este documento describe la estrategia implementada en el jugador autónomo **SmartPlayer** para el juego HEX. El agente combina una búsqueda adversarial avanzada con heurísticas precisas, utilizando técnicas como Negamax con poda alfa-beta, Principal Variation Search (PVS), Late Move Reductions (LMR), tabla de transposición con hashing Zobrist, killer moves, history heuristic, y una heurística de dos distancias basada en 0-1 BFS. Para tableros grandes ( $19 \times 19$  o más) se emplea un fast-path heurístico que evita la explosión combinatoria en la fase inicial. El sistema se ajusta dinámicamente al tiempo disponible, garantizando respuestas en menos de 5 segundos por jugada.

## 1. Introducción

HEX es un juego de estrategia para dos jugadores en un tablero hexagonal de tamaño  $N \times N$ . El jugador 1 (rojo) debe conectar los lados izquierdo y derecho; el jugador 2 (azul) debe conectar los lados superior e inferior. Dada la alta complejidad combinatoria (el árbol de juego crece exponencialmente), se requiere una inteligencia artificial eficiente que combine búsqueda exhaustiva con heurísticas informadas.

El jugador implementado, **SmartPlayer**, hereda de la clase base **Player** y utiliza un motor de búsqueda moderno que incorpora técnicas de vanguardia. A continuación se detalla su arquitectura y estrategia.

## 2. Arquitectura general

La clase principal **SmartPlayer** define el método `play(board)` que se invoca en cada turno. Su funcionamiento se resume en los siguientes pasos:

1. **Identificación del jugador:** mediante `_identify_player` se determina si el motor es el jugador 1 o 2 (busca atributos como `player_id` o, en su defecto, cuenta piedras).
2. **Fast-path para tableros grandes:** si  $N > 18$  y más del 55 % de las celdas están vacías, se usa una heurística rápida (`_play_large_board`) que evita la búsqueda exhaustiva.

3. **Apertura:** si es la primera jugada, se aplica una apertura predefinida (centro o respuesta al centro).
4. **Búsqueda completa:** en caso contrario, se construye la tabla de vecinos, se calcula un presupuesto de tiempo adaptativo y se instancia un `SearchEngine` que realiza la búsqueda iterativa con todas las optimizaciones.

El motor de búsqueda (`SearchEngine`) se crea de nuevo en cada turno, garantizando la idempotencia.

### 3. Algoritmo de búsqueda: Negamax con podas avanzadas

#### 3.1. Negamax y poda alfa-beta

El núcleo es el algoritmo **Negamax**, que simplifica la recursión minimax usando la propiedad

$$\text{valor}(s) = - \max_a \text{valor}(s').$$

Se implementa en la función `_negamax`, que recibe la profundidad, los límites  $\alpha$  y  $\beta$ , y el jugador que debe mover. La poda alfa-beta elimina ramas cuando  $\alpha \geq \beta$ , reduciendo drásticamente el número de nodos explorados.

#### 3.2. Principal Variation Search (PVS)

En la raíz y en nodos internos se aplica **PVS**. El primer movimiento se explora con ventana completa  $(-\beta, -\alpha)$ . Los movimientos restantes se prueban primero con una ventana nula  $(-\alpha - 1, -\alpha)$ . Si el resultado cae dentro de la ventana, se re-explora con ventana completa. Esto acelera la búsqueda en órdenes de movimientos bien ordenados.

#### 3.3. Late Move Reductions (LMR)

Para movimientos que no son el primero, no son el movimiento de la PV, ni son killers, se aplica una **reducción de profundidad** de 1 nivel (si la profundidad  $\geq 4$  y el índice del movimiento  $\geq 4$ ). Si la búsqueda reducida mejora  $\alpha$ , se re-explora a profundidad completa. Esta técnica permite explorar más nodos sin sacrificar precisión.

#### 3.4. Tabla de transposición y Zobrist hashing

Cada posición se identifica mediante un **hash Zobrist** de 64 bits. La tabla `zt` se genera con una semilla fija (42), asegurando determinismo. En cada movimiento, el hash se actualiza incrementalmente con XOR.

La tabla de transposición (`self.tt`) almacena entradas con la profundidad, el tipo de bound (exacto, lower, upper), el valor y el mejor movimiento. Se aplica una política de reemplazo **depth-preferred**: una entrada más profunda nunca es sobrescrita por una más

superficial. Si la tabla alcanza `TT_MAX_ENTRIES` (500000), se limpia por completo para mantener la memoria acotada.

### 3.5. Killer moves e historia

- **Killer moves:** se almacenan dos movimientos por profundidad que causaron un corte beta. En el ordenamiento, estos reciben bonificaciones altas (`KILLER_PRIMARY_BONUS=50000`, `KILLER_SECONDARY_BONUS=40000`).
- **History heuristic:** cada celda y jugador acumula un contador (`history[r][c][player]`) que se incrementa con  $depth^2$  cada vez que el movimiento causa un corte beta. En el ordenamiento se suma `history[r][c][player] × HISTORY_MULTIPLIER (2)`.

### 3.6. Profundización iterativa y ventanas de aspiración

La búsqueda se realiza con **profundización iterativa**: se comienza con profundidad 1 y se incrementa hasta agotar el tiempo. Después de la primera iteración, se guarda la puntuación obtenida. En iteraciones posteriores, se usa una **ventana de aspiración** alrededor de esa puntuación ( $last\_score \pm 300$ ). Si la búsqueda falla (el valor queda fuera de la ventana), se vuelve a ejecutar con ventana completa. Esto acelera la convergencia.

## 4. Heurística de evaluación: two-distance con 0-1 BFS

### 4.1. 0-1 BFS para distancias

Para evaluar una posición, se calcula la distancia mínima que cada jugador necesita para conectar sus dos lados. El grafo tiene costos binarios: 0 para celdas propias, 1 para celdas vacías, e infinito para celdas del oponente. Se utiliza **0-1 BFS** (cola doble) en lugar de Dijkstra, obteniendo complejidad  $O(V+E)$  frente a  $O((V+E) \log V)$ . La función `bfs01_distance` devuelve la distancia mínima para un jugador.

### 4.2. Búsqueda bidireccional y conteo de caminos

Para una evaluación más precisa se emplea `two_distance`, que realiza dos búsquedas 0-1 BFS: una desde el borde inicial (`bfs01_full`) y otra desde el borde final (`bfs01_reverse`). La suma  $d_{fwd}[r][c] + d_{rev}[r][c]$  indica la distancia del camino más corto que pasa por la celda  $(r, c)$ . El mínimo de estas sumas en el borde opuesto es la distancia de conexión.

Además, se cuenta el número de celdas (excluyendo las del oponente) que pertenecen a algún camino óptimo (aquellas cuya suma es igual a la distancia mínima). Este valor, `on_path`, mide la redundancia: más caminos implican una posición más robusta.

### 4.3. Fórmula de evaluación

- **Nodos internos profundos** (`evaluate_fast`):

$$\text{score} = (d_{\text{rival}} - d_{\text{propio}}) \times 200$$

Si el tablero es pequeño ( $\leq 11$ ) y las distancias son iguales, se añade la diferencia de caminos con peso 10.

■ **Nodos hoja** (`evaluate_board`):

$$\text{score} = (d_{\text{rival}} - d_{\text{propio}}) \times 200 + (\text{caminos}_{\text{propio}} - \text{caminos}_{\text{rival}}) \times 10$$

Cuando un jugador tiene camino y el otro no, la función devuelve  $\pm 500\,000$ ; cuando se detecta una victoria, retorna  $1\,000\,000$ .

## 5. Ordenamiento de movimientos

En `_order_moves` se asigna una puntuación a cada celda vacía. Los criterios, en orden de prioridad:

1. **Movimiento de la PV**: se coloca al principio con puntuación  $-10\,000\,000$ .
2. **Killer moves**: si coincide con el primer o segundo killer del nivel, se añaden bonificaciones  $50\,000$  o  $40\,000$ .
3. **Historia**:  $\text{history}[r][c][player] \times 2$ .
4. **Conectividad local**: cada vecino propio suma  $12$ ; cada vecino rival suma  $7$ .
5. **Borde conectivo**: si la celda está en el borde del jugador y tiene al menos un vecino propio, se añade  $5\,000$ .
6. **Puentes virtuales**: si la celda completa un puente (según tabla precomputada) y ambas portadoras están vacías, se añade  $20$ .
7. **Centralidad**: se penaliza la distancia al centro:  $(size - dist\_center) \times 3$ .
8. **Alineación al eje**: para jugador 1 se bonifica estar cerca del centro horizontal; para jugador 2, cerca del centro vertical. Peso  $1,5$ .
9. **Desempate determinista**: se aplica una pequeña perturbación basada en la celda y el jugador para romper empates de forma reproducible.

Las puntuaciones se ordenan de mayor a menor.

## 6. Detección de victorias con DSU incremental

### 6.1. Estructura DSU con rollback

Se utiliza una estructura **Union-Find** con capacidad de deshacer cambios (`checkpoint/rollback`). Cada celda tiene un identificador  $r \cdot size + c$ . Se añaden cuatro nodos virtuales:

$$LEFT = N^2, \quad RIGHT = N^2 + 1, \quad TOP = N^2 + 2, \quad BOTTOM = N^2 + 3.$$

La función `dsu_check_win` comprueba si *LEFT* y *RIGHT* están conectados (jugador 1) o *TOP* y *BOTTOM* (jugador 2).

## 6.2. Actualización incremental

Al colocar una piedra, `dsu.add_stone` une la celda con los bordes correspondientes y con sus vecinos del mismo jugador. Antes de hacer un movimiento se guarda un checkpoint; después de explorar, se restaura con `rollback`.

## 6.3. Búsqueda de victorias inmediatas

Antes de la búsqueda, `_find_immediate_win` prueba cada celda vacía: si al colocar la piedra propia el DSU detecta victoria, esa celda se juega de inmediato. También se verifica si el oponente tiene una victoria inmediata y se bloquea (`_winning_cells_for_player`). Si hay un solo bloqueo, se juega directamente.

## 7. Gestión del tiempo

El límite global es `TIME_BUDGET = 4.0` segundos. Por seguridad, se aplica un factor `TIME_SAFETY_FACTOR = 0.9`, resultando en un límite efectivo que nunca supera los 3.6 segundos.

- **Presupuesto por turno:** se calcula en `_compute_turn_budget` según la proporción de celdas vacías. En fases tempranas (progreso  $\leq 0.2$ ) se asigna el 65 % del presupuesto; en fase media (0.2–0.6) el 78 %; en fase tardía ( $\geq 0.6$ ) el 88 %. Para tableros pequeños ( $\leq 7$ ) se añade un pequeño bono.
- **Verificación durante la búsqueda:** cada 1024 nodos (`self.nodes & TIME_CHECK_MASK == 0`) se comprueba el tiempo con `_check_time`. Si se excede, se lanza una excepción `TimeoutError` que se captura en el bucle de profundización iterativa, interrumpiendo la búsqueda y retornando el mejor movimiento encontrado hasta ese momento.

## 8. Apertura y juego en tableros grandes

### 8.1. Apertura

Si el tablero está casi vacío ( $empty\_count \geq size^2 - 1$ ), se juega:

- Centro ( $\frac{size}{2}, \frac{size}{2}$ ) si está libre.
- Si el oponente ya ocupó el centro, se evalúan las celdas adyacentes según el jugador. Para jugador 1 se priorizan celdas con  $nc$  cercano al centro; para jugador 2 se priorizan celdas con  $nr$  cercano al centro.

### 8.2. Fast-path para tableros grandes

Cuando  $size > 18$  y la fracción de celdas vacías es mayor a `LARGE_BOARD_SPARSE_PHASE (0.55)`, se activa un fast-path heurístico (`_play_large_board`). Este:

- Toma como candidatas las celdas vecinas a piedras existentes (hasta radio 2).
- Las puntúa según vecinos propios/rivales, centralidad y alineación al eje.
- Retorna la mejor celda sin realizar búsqueda en profundidad.

Esta estrategia evita la explosión combinatoria en la fase inicial de tableros 19×19 y superiores.

## 9. Optimizaciones adicionales

Todas las optimizaciones que se describen a continuación trabajan con estructuras de datos que se crean de nuevo en cada llamada a `play()` (o, en el caso de `SearchEngine`, en cada turno). De esta forma se garantiza que no se conserva información entre partidas.

- **Caché de vecinos:** La función `build_neighbor_table` precalcula los vecinos para tableros de tamaño  $\leq 100$  y devuelve una nueva lista bidimensional. Para tableros mayores de 100, retorna una nueva instancia de `LazyNeighborTable`, que almacena internamente un diccionario `_cache` para recordar los vecinos ya calculados durante la misma partida. Esta caché es local a la instancia y se destruye al terminar la partida.
- **Caché de evaluación:** Dentro de `_negamax`, las evaluaciones de los nodos hoja se guardan en `eval_cache` (un diccionario que pertenece a la instancia de `SearchEngine`) utilizando una clave derivada del hash Zobrist. Esta caché se limpia automáticamente cuando supera `EVAL_CACHE_MAX_ENTRIES` (200 000 entradas) y, lo más importante, se recrea desde cero en cada nuevo turno porque `SearchEngine` es una instancia nueva.
- **Eliminación eficiente de celdas vacías:** Se mantienen dos estructuras paralelas –una lista `empty_cells` y un diccionario `empty_pos`– que permiten eliminar y restaurar movimientos en tiempo constante durante la búsqueda. Ambas son locales a la llamada a `search()` y no persisten más allá de la evaluación de un movimiento.
- **Poda de celdas inferiores:** La función `_prune_inferior_cells` descarta celdas aisladas y muy alejadas del centro cuando el número de candidatos supera `max_branch`. Esta decisión se toma sobre la lista de candidatos generada en cada nodo de búsqueda y no guarda ningún estado entre nodos.
- **Variables globales de compatibilidad:** En el archivo existen tres variables globales (`_NEIGHBOR_TABLE_CACHE`, `_BRIDGE_CACHE`, `_ZOBRIST_CACHE`) que son instancias de la clase `_CompatNoStateCache`. Estas variables solo están presentes para satisfacer la interfaz esperada por el entorno de torneo (que llama a `clear()` entre partidas). No almacenan ninguna tabla real y su método `clear()` no tiene efecto. Todas las estructuras de datos que realmente se usan (tablas de vecinos, puentes, Zobrist, etc.) se crean localmente en cada llamada a `play()` y nunca se asignan a dichas variables globales.

## 10. Conclusiones

La IA implementada combina técnicas modernas de búsqueda adversarial con una heurística precisa basada en distancias en grafos. La arquitectura híbrida (búsqueda exhaustiva + fast-path) la hace competitiva tanto en tableros pequeños como en tableros grandes. El control de tiempo adaptativo garantiza que nunca se exceda el límite de 5 segundos por jugada.